

Estructuras de selección

La máquina expendedora de la Facultad, en dos clases, cada una en su archivo:

`Producto` y `Expendedora`. Tú escribes sus atributos, sus constructores y sus métodos, y en cada uno decides con `if`, `switch` o el ternario.

[Descargar los archivos \(.zip\)](#)

En una línea

Corres `P5Seleccion.java` y ves dieciocho **FALLA**. Completas `Producto.java` y luego `Expendedora.java` donde dice `TODO`, vuelves a correr, y terminas cuando la tabla diga **18 DE 18**.

ARCHIVO	QUÉ ES	QUÉ HACES
<code>P5Seleccion.java</code>	El <code>main</code> con las 18 pruebas	Correrlo. No se toca
<code>Moneda.java</code>	El enum de las monedas	Nada: ya está escrito
<code>Producto.java</code>	Paso 1: pruebas 1 a 6	Completar los <code>TODO</code>
<code>Expendedora.java</code>	Paso 2: pruebas 7 a 18	Completar los <code>TODO</code>

Los cuatro van **juntos en la misma carpeta** (en IntelliJ, en `src`).

Parte 0 · Qué se entrega y cómo se califica

QUÉ	DETALLE
Cuándo	Viernes 2 de octubre, 23:59. Se empieza en el laboratorio y ahí mismo se resuelven las dudas.

Con quién	En parejas, o de tres si alguien se queda sin pareja. Se reparten el trabajo como quieran, pero cada integrante sube la entrega completa en su propia carpeta.
Dónde	Tu rama <code>entregas_apellido_nombre</code> , carpeta <code>entregas/apellido_nombre/p05/</code> . Solo push, sin pull request.
Qué archivos	Los cuatro <code>.java</code> , todos los <code>.class</code> que salen al compilar y una bitácora en PDF con: los integrantes (nombre y número de cuenta), qué se les complicó y cómo lo resolvieron, y la captura de la terminal con la tabla final.
Cómo se califica	Cuántas de las 18 pasan, que uses la estructura que pide cada método y cada constructor y que el código sea tuyo. Si algo no les sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Parte 1 · Un ejemplo completo, con otra clase

Así se ve una clase con todo lo que vas a escribir hoy. No es la de la práctica: es un `Alumno`, para que veas la forma y luego la apliques a `Producto` y `Expendedora`.

```

public class Alumno {

    // CONSTANTE: static final, en MAYÚSCULAS, no cambia nunca
    static final int EDAD_MINIMA = 17;

    // ATRIBUTO STATIC: uno solo, compartido por todos los alumnos
    static int totalAlumnos = 0;

    // ATRIBUTOS: cada alumno tiene los suyos
    String nombre;
    int edad;

    // CONSTRUCTOR: mismo nombre que la clase, sin tipo de retorno
    Alumno(String nombre, int edad) {
        this.nombre = nombre;           // this.atributo = parametro
        if (edad < 0) {                  // if / else: validar lo que
llega
            this.edad = 0;
        } else {
            this.edad = edad;
        }
        totalAlumnos++;                 // cuenta a este alumno
    }

    // MÉTODO con escalera if / else if / else
    String etapa() {
        if (edad < 18) {
            return "MENOR";
        } else if (edad < 30) {
            return "JOVEN";
        } else {
            return "ADULTO";
        }
    }

    // MÉTODO con ternario: condicion ? siTrue : siFalse
    String puedeInscribirse() {

```

```

        return edad >= EDAD_MINIMA ? "SI" : "NO";
    }

    // MÉTODO static: se llama con el nombre de la clase
    static int getTotalAlumnos() {
        return totalAlumnos;
    }
}

```

Y así se usa:

```

Alumno a = new Alumno("Ana", 19);
Alumno b = new Alumno("Beto", -3);

a.etapa()           // "JOVEN"
a.puedeInscribirse() // "SI"
b.etapa()           // "MENOR"   (su edad se guardó como 0)
Alumno.getTotalAlumnos() // 2

```

El switch clásico, con otro ejemplo

```

int numero;
switch (dia.trim().toUpperCase()) { // " martes " se vuelve "MARTES"
    case "LUNES":
        numero = 1;
        break; // sin break, sigue al case de
abajo
    case "MARTES":
        numero = 2;
        break;
    default: // ningún case coincidió
        numero = -1;
}
return numero;

```

Con un **enum** es igual. El `case` se escribe corto, `case DIEZ:`, sin el nombre del enum.

Parte 2 · Lo que vas a escribir

La máquina vende tres productos, recibe monedas, guarda el saldo y al comprar entrega el producto y el cambio. Le caben como máximo 10 piezas de cada producto.

CÓDIGO	PRODUCTO	PRECIO
A1	papas	\$18
A2	refresco	\$22
B1	chocolate	\$15

CLASE	PARTE	QUÉ HACE	USA
Producto	Atributos	Una constante con la capacidad (10), y de cada producto: nombre, precio y piezas	
	Constructor <code>Producto(String, int, int)</code>	Piezas menores que 0 se guardan como 0; mayores que 10, como 10	if / else if / else
	<code>getPrecio()</code> · <code>getPiezas()</code>	Regresan el atributo	
	<code>hayPiezas()</code>	true si queda al menos una	
	<code>sacarUna()</code>	Quita una pieza, pero nunca baja de 0	if
	<code>existencia()</code>	"AGOTADO", "ULTIMAS PIEZAS" (1 o 2) o "DISPONIBLE"	if / else if / else
Expendedora	Atributos	Un contador <code>static</code> de ventas de todas las máquinas, y de cada máquina: ubicación, saldo y sus tres <code>Producto</code>	
	Constructor <code>Expendedora(String,</code>	Crea sus tres productos con <code>new</code> . Ubicación en blanco: se	ternario

<code>int)</code>	guarda <code>"SIN UBICACION"</code>	
<code>buscar(String)</code>	El <code>Producto</code> de ese código, o <code>null</code> . <code>" a2 "</code> vale igual que <code>"A2"</code>	switch clásico
<code>insertar(Moneda)</code>	Suma la moneda al saldo: UNO, DOS, CINCO o DIEZ	switch sobre el enum
<code>comprar(String)</code>	En este orden: <code>"CODIGO INVALIDO"</code> , <code>"AGOTADO"</code> , <code>"FALTAN \$3"</code> o <code>"ENTREGADO, CAMBIO \$15"</code>	if / else if / else
<code>estado()</code>	<code>"VACIA"</code> si ningún producto tiene piezas; si no, <code>"DISPONIBLE"</code>	ternario
<code>etiqueta()</code>	<code>"Edificio Q (DISPONIBLE)"</code>	
<code>cancelar()</code> · <code>getVentasTotales()</code>	Devuelve el saldo y lo deja en 0 · las ventas de todas las máquinas	

Qué debe pasar: ejemplos

SI EL MAIN HACE ESTO	TU CÓDIGO DEBE DAR
<code>new Producto("papas", 18, 15).getPiezas()</code>	<code>10</code> (solo caben 10)
<code>new Producto("chicles", 5, -4).getPiezas()</code>	<code>0</code>
<code>new Producto("gomitas", 8, 2).existencia()</code>	<code>"ULTIMAS PIEZAS"</code>
gomitas con 2 piezas: <code>sacarUna()</code> tres veces	2, 1, 0 y se queda en <code>0</code>
<code>new Expendedora(" ", 0).etiqueta()</code>	<code>"SIN UBICACION (VACIA)"</code>
<code>m.buscar(" a2 ").getPrecio()</code>	<code>22</code>
<code>m.buscar("Z9")</code>	<code>null</code>

Una compra, paso a paso

```
Expendedora m = new Expendedora("Edificio Q", 15); // 10 piezas de
cada producto, saldo 0

m.insertar(Moneda.DIEZ);           // saldo 10
m.insertar(Moneda.CINCO);          // saldo 15
m.comprar("A1");                    // "FALTAN $3"    las papas cuestan 18;
el saldo se queda en 15

m.insertar(Moneda.DOS);             // saldo 17
m.insertar(Moneda.UNO);             // saldo 18
m.comprar("A1");                    // "ENTREGADO, CAMBIO $0"    papas: 9,
saldo: 0, ventas: 1

m.comprar("C3");                    // "CODIGO INVALIDO"

m.insertar(Moneda.DIEZ);            // saldo 10
m.insertar(Moneda.DIEZ);            // saldo 20
m.insertar(Moneda.DIEZ);            // saldo 30
m.comprar("B1");                    // "ENTREGADO, CAMBIO $15"    chocolate
cuesta 15; ventas: 2
```

En una máquina creada con 0 piezas, `comprar("B1")` da `"AGOTADO"` aunque tenga saldo: primero se pregunta si hay piezas y después si alcanza el dinero.

Parte 3 · Cómo sabes que quedó

```
javac P5Seleccion.java           # compila también Moneda, Producto y
Expendedora
java P5Seleccion
```

CUANDO YA ESTÁ

PRODUCTO			
1	papas con 15 piezas	10	10 ok
...			
18	getVentasTotales()	2	2 ok

18 DE 18

Listo. Toma la captura de esta tabla y pegala en tu bitacora.

Lo que más truena

- **La 7 da `null (DISPONIBLE)`**: te faltó `this.` al guardar la ubicación. Compila, pero no guarda nada.
- **Muchas dan truena: `NullPointerException`**: un `Producto` de la máquina se quedó en `null`. Te faltó crearlo con `new` en el constructor.
- **La 9 truena y la 10 da `true, false`**: no limpiaste el código con `trim()` y `toUpperCase()`.
- **La 16 da `FALTAN $5`**: el orden de tu escalera. Primero se pregunta si hay piezas, luego si alcanza el dinero.

La entrega, paso a paso

```
git checkout entregas_apellido_nombre
git pull origin main
mkdir -p entregas/apellido_nombre/p05
# copia aquí los cuatro .java, todos los .class y la bitácora en PDF
git add entregas/apellido_nombre/p05
git status
git commit -m "P5: estructuras de seleccion"
git push
```

Al compilar salen varios `.class` (`P5Seleccion`, `Producto`, `Expendedora`, `Moneda` y quizá `Expendedora$1`). Súbelos todos. Si no aparecen en `git status`, usa `git add -f`.

**Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería,
UNAM**

Prof. Oscar Manuel Ruiz Hurtado · Práctica 5 · Estructuras de selección (MADO-21, guía 5) ·
septiembre de 2026